Module 3 - Integrating Quality Assurance (QA) in the SDLC and Secure Development (SSDLC), with ISO/IEC 27001 A.14

Contents

Learning Objectives 3
Why This Topic Matters 3
Core Concepts 4
QA Checkpoints in Each SDLC Phase 11
ISO/IEC 27001 Annex A.14.....17
PHP Demo: Registration + Login with Hashing + Basic Validation 18

Learning Objectives

By the end of this lesson, students will be able to:

1. **Define** SDLC, QA, and SSDLC in simple terms and give **one real-life analogy** for each.
2. **Identify** the common SDLC phases and **describe at least one QA checkpoint** in each phase.
3. **Explain** how security fits into development by outlining a **Secure SDLC flow** (where security checks happen and why).
4. **Map** the intent of **ISO/IEC 27001 Annex A.14** to practical development activities (e.g., requirements, design, testing, change control).
5. **Create** a simple SDLC diagram for a sample project and **label QA + security checkpoints** correctly.
6. **Apply** a checklist to a mini-case and **recommend improvements** (what to test, when to review, what security controls to add).

Why This Topic Matters

A simple story

Imagine you're building a **bridge**.
If you only check for problems after cars start using it, you might discover cracks too late.
The fix becomes expensive, dangerous, and embarrassing.

Software is similar:

- Bugs can cause crashes and lost data.
- Weak security can lead to hacking, fraud, leaks, and real harm to people. • Fixing problems **late** costs much more than preventing them **early**.

QA is how we build software that works reliably.
Secure development is how we build software that stays safe even when people try to break it.

Core Concepts

What is SDLC?

Software Development Life Cycle (SDLC) is the step-by-step process teams follow to build software from idea to release and maintenance.

Typical phases:

1) Planning

What happens

- Identify the problem/opportunity and the goal of the system
- Define scope (what's included / excluded)
- Estimate time, budget, people, risks
- Decide if the project is feasible and worth doing

Typical output documents

- **Project Charter** (purpose, scope, stakeholders, high-level timeline) • **Feasibility Study / Business Case** (cost–benefit, technical feasibility, options)
- **Project Plan** (schedule, resources, risk plan)
- **High-level Risk Register**

2) Requirements

What happens

- Gather needs from users/stakeholders (interviews, workshops, observation)
- Convert needs into clear, testable requirements
- Define functional and non-functional requirements (performance, security, usability, availability)
- Agree on scope and acceptance criteria

Typical output documents

- **SRS (Software Requirements Specification)** *(classic / formal)*
- **BRD (Business Requirements Document)** *(business-focused)*
- **User Stories + Acceptance Criteria** *(Agile style)*
- **Use Case Document** / **Requirements Traceability Matrix (RTM)** *(often added to track req → tests)*

3) Design

What happens

- Plan how the system will work to meet requirements
- Decide architecture (layers, modules, integrations)
- Design database, APIs, UI/UX flows
- Define security approach, error handling, logging, performance considerations

Typical output documents

- **SDD (Software Design Document)** / **System Design Specification**
- **Architecture Diagram(s)** (context diagram, component diagram, deployment diagram)
- **Database Design** (ERD, data dictionary)
- **UI Wireframes / Mockups**
- **API Specification** (e.g., OpenAPI/Swagger)

4) Implementation (Coding)

What happens

- Developers build features based on the design
- Code reviews, branching strategy, CI builds
- Create initial technical documentation and configs

Typical output documents/artifacts

- **Source Code Repository** (main deliverable)
- **Build/Release Artifacts** (packages, containers, compiled binaries) •

Technical Notes / Developer Docs (setup instructions, coding standards)

- **Configuration Files** and **Environment Setup Guides**

5) Testing

What happens

- Verify the system works and matches requirements
- Test levels: unit, integration, system, regression, security, performance,

UAT • Log defects and confirm fixes

Typical output documents

- **Test Plan**
- **Test Cases / Test Scripts**
- **Test Report / Test Summary Report**
- **Bug/Defect Reports**
- **UAT Sign-off Document** *(if applicable)*
- **RTM updated** (requirements mapped to tests + results)

6) Deployment

What happens

- Prepare production environment
- Release the system (manual or automated)
- Data migration, cutover plan, rollback plan
- Train users and hand over to operations/support

Typical output documents

- **Deployment Plan / Release Plan**
- **Release Notes / Change Log**
- **Installation Guide / Runbook**
- **Rollback Plan**
- **Training Materials / User Manual** *(often delivered here)*

7) Maintenance / Improvement

What happens

- Monitor system health, handle incidents and bugs
- Patch vulnerabilities, optimize performance
- Add enhancements based on feedback
- Do periodic reviews and updates

Typical output documents

- **Maintenance Plan**
- **Incident Reports / Postmortems**
- **Patch Notes / Version History**
- **Change Requests / Enhancement Backlog**

- **Updated Documentation** (SRS/SDD/runbooks updated as system evolves)

What is QA?

QA (Quality Assurance) is the *way a team prevents problems* by making sure the **process of building software** is organized, consistent, and followed—so the final software is more likely to be high-quality.

Think of it like this:

- **QA = prevention** ("How do we build it the right way so fewer bugs

happen?") • **QC = detection** ("Let's test the finished thing and catch bugs.")

Simple analogy

If you're cooking:

- **QA** is having a clean kitchen, correct recipe, proper measurements, and a checklist.
- **QC** is tasting the food at the end and checking if it's cooked

right. **What QA does in software**

QA includes planned activities such as:

- Setting **standards** (coding rules, documentation rules)

- Doing **reviews** (requirements review, design review, code

review) • Making sure there is a clear **test plan** and **acceptance**

criteria

- Ensuring **process steps are followed** (e.g., "no feature is merged without review + tests")

- Supporting **continuous testing** (CI checks, automated tests)

- Tracking issues and ensuring fixes are verified (**defect**

management) **Common QA outputs**

- **QA Plan** (what quality activities will be done, when, and who does

them) • **Test Strategy / Test Plan**

- **Checklists** (requirements checklist, code review checklist, release checklist)
- **Standards / Guidelines** (coding standards, definition of done)
- **Metrics & Reports** (defect trends, test coverage, pass/fail summaries)

What is Secure SDLC (SSDLC)?

SSDLC is building software with "security thinking" from day 1—so the app is harder to hack, protects data, and fails safely.

What changes in each SDLC phase (with typical outputs)

1) Planning (security starts here)

Security focus

- Decide what needs protection (data, accounts, money, critical services)
- Identify laws/standards you must follow (privacy, government rules, etc.)
- Plan security budget, tools, and roles

Common outputs

- **Security Requirements (high-level)**
- **Compliance checklist** (if applicable)
- **Initial Risk Register**

2) Requirements

Security focus

- Add **security requirements** (auth, access control, encryption, logging, privacy) • Define **non-functional security goals** (availability, confidentiality, integrity) • Agree on **abuse cases** (how an attacker might misuse the system)

Common outputs

- **Security Requirements Specification** (or security section in SRS) • **Abuse/Misuse Cases**

- **Updated Risk Register**

3) Design

Security focus

- Threat modeling: “How can this be attacked?”
- Design secure architecture (least privilege, secure APIs, proper session handling) • Plan secure storage, encryption, key management, audit logs

Common outputs

- **Threat Model** (often using STRIDE)
- **Security Architecture / Design Review notes**
- **Data Flow Diagram (DFD) + trust boundaries**
- **Secure design decisions** (controls mapped to risks)

4) Implementation (Coding)
Security focus

- Secure coding practices (validate input, safe auth, protect secrets)
- Use dependency scanning (avoid vulnerable libraries) • Code review focused on security

Common outputs

- **Secure Coding Checklist evidence**
- **Code Review records**
- **SAST results** (static code scan report)
- **Dependency scan / SBOM** (software bill of materials)

5) Testing

Security focus

- Security testing alongside functional testing
- Dynamic testing (DAST), API testing, auth testing
- Pen testing (for higher-risk systems)

Common outputs

- **Security Test Plan / Test Cases**
- **Vulnerability reports** (DAST, SAST, scans)
- **Penetration Test Report** (if performed)
- **Fix verification report / retest results**

6) Deployment

Security focus

- Secure configuration (hardening), least privilege access
- Secrets management (no passwords in code)
- Monitoring, logging, alerting enabled
- Secure release and rollback plan

Common outputs

- **Secure Deployment Checklist**
- **Hardening Guide / Runbook**
- **Access control matrix** (who can access what in prod)
- **Logging/Monitoring configuration evidence**

7) Maintenance / Improvement

Security focus

- Patch vulnerabilities regularly
- Monitor alerts and respond to incidents

- Review access, rotate keys, update dependencies

Common outputs

- **Patch/Vulnerability Management logs**
- **Incident Response reports**
- **Periodic security review reports**
- **Change requests with security impact notes**

QA Checkpoints in Each SDLC Phase

QA checkpoints are scheduled activities (reviews, validations, audits, and test preparations) done throughout the SDLC to ensure both the process and the deliverables are high quality—so defects are prevented early, not only found at the end.

Phase 1: Planning

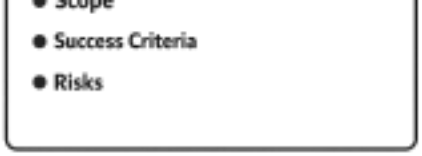
Goal: Decide what we're building and why.

QA Checkpoints

- Define success criteria (What does "good" look like?)
- Identify stakeholders (who will use it?)
- Choose a development approach and basic standards (naming conventions, documentation habits)

Security Checkpoints

- Identify sensitive data early (names, passwords, grades, money, health info)
- Define security goals (e.g., "Only students can see their grades")



A visual summary box used to present the project's key points—scope, success criteria, and risks—in a single view for quick alignment and reference.

Phase 2: Requirements

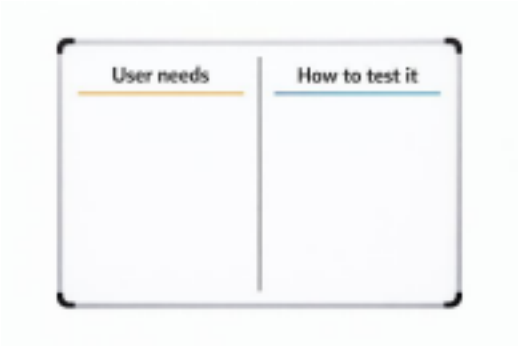
Goal: Clearly describe what the system must do.

QA Checkpoints

- Requirements must be **clear, testable, and not contradictory**
- Create acceptance criteria (how we will confirm it works)
- Build simple user stories (e.g., "As a student, I can log in…")

Security Checkpoints

- Add security requirements:
  - Authentication (login)
  - Authorization (what each role can access)
  - Data privacy
- Identify misuse cases: "How could someone abuse this feature?"



*A **"User needs vs. How to test it" board** is a simple QA planning visual that **links what the user expects** (requirements) to **how the team will verify it works** (test approach). It helps ensure that every user need has a clear, testable proof—so nothing important is built without being checked.*

Phase 3: Design

Goal: Plan how the system will work internally.

QA Checkpoints

- Design review meeting (does it satisfy requirements?)
- Define test strategy early (unit, integration, system tests)
- Consider usability (is it simple to use?)

Security Checkpoints

- Threat modeling (beginner version: "What can go wrong?")
- Decide security controls:
    - Password hashing
    - Input validation
    - Secure session handling
- Plan logging and error handling (don't show secret details in errors)

Simple system architecture with security gates. *The diagram shows the interaction flow from the user to the application and then to the database, with security controls applied at key points: user authentication through login and authorization through permission checks before database access.*

Phase 4: Implementation (Coding)

Goal: Write the software.

QA Checkpoints

- Code reviews (another person checks your work)
- Unit tests (small tests for functions)
- Static checks (linting, formatting)

Security Checkpoints

- Use safe libraries
- Avoid hard-coded secrets (passwords in code)
- Validate inputs (never trust user input)
- Follow secure coding guidelines

Phase 6: Deployment

Goal: Release to real users.

QA Checkpoints

- Release checklist
- Rollback plan (what if release fails?)
- Monitor errors

Security Checkpoints

- Secure configuration:
  - HTTPS enabled
  - Minimal privileges for services
  - Safe environment variables
- Confirm backups and logging

*The figure illustrates a three-stage software release flow moving from **Development (Dev)** to **Staging** and finally to **Production**. Between each environment is a **Go/No-Go gate**, shown as*

a decision checkpoint, where release readiness is evaluated. Only when the criteria are met (Go) does the build proceed to the next stage; otherwise (No-Go) the release is paused for fixes or further testing. This highlights controlled promotion of changes to reduce risk before reaching the production environment.

Phase 7: Maintenance / Improvement

Goal: Keep software working and safe over time.

QA Checkpoints

- Bug triage (prioritize fixes)
- Continuous improvement (post-release reviews)
- Update documentation

Security Checkpoints

- Patch updates (libraries, frameworks)
- Security incident handling (what to do if breach happens)
- Regular reviews of access permissions

*The figure shows a circular, clockwise arrow loop representing an ongoing cycle of system maintenance and enhancement. The cycle begins with **Monitor**, where performance and issues are observed, continues to **Improve**, where findings are analyzed and solutions are planned, and proceeds to **Update**, where changes and fixes are implemented. The loop format emphasizes that after updates are applied, monitoring continues to ensure sustained quality and ongoing improvement.*

ISO/IEC 27001 Annex A.14

ISO/IEC 27001 is a standard for managing information security.
Annex A.14 focuses on **security in system acquisition, development, and**

maintenance. Think of A.14 as saying:

“When you build or change software, security requirements must be included, applied, tested, and controlled—so changes don’t quietly introduce risks.”

What A.14 looks like in practice (simple mapping)

A.14 → Practical actions

- **Security requirements exist**
  → You write security requirements (login rules, access control, privacy)
- **Systems are built securely**
  → You use secure coding practices + reviews
- **Changes are controlled**
  → You track changes, approvals, testing, rollback plans
- **Testing includes security**
  → You verify access restrictions and handle inputs safely
- **Development environment is protected**
  → Not everyone has admin access; secrets aren’t exposed; code repositories are protected

PHP Demo: Registration + Login with Hashing + Basic Validation

```
<?php

$users = [];


/** Register using password_hash (recommended). */

function register_user(string $email, string $password, array &$users):

string {  if (strpos($email, '@') === false) {

 return "Invalid email format.";

 }

 if (strlen($password) < 8) {
```

```
 return "Password must be at least 8 characters.";
 }

 // Strong hashing with salt built-in (recommended)
 $users[$email] = password_hash($password, PASSWORD_DEFAULT);

 return "Registered successfully.";
}

/** Login using password_verify (recommended). */
function login_user(string $email, string $password, array $users):
string {  // Generic error message either way
 if (isset($users[$email]) && password_verify($password,
$users[$email])) {  return "Login successful.";
 }
 return "Invalid login.";
}

echo register_user("student@example.com", "mypassword", $users) .
PHP_EOL; echo login_user("student@example.com", "mypassword",
$users) . PHP_EOL; echo login_user("student@example.com", "wrongpass",
$users) . PHP_EOL; ?>
```

What is this code trying to do?

This code shows a basic idea of how websites do:

1. **Register** a user (save their email + password)
2. **Login** a user (check if the email + password is correct)

But it does it in a **secure way** by **not saving the real password**.

Step 1: A place to store users

$users = [];

What this means (simple):

- This is an empty list (array).
- It will store users like this:

email → password_hash

Example:

- "student@example.com" → "a long scrambled hash like

$2y$10$..." **Important:**

This is only for **demo**.
Real websites store this in a **database**.

Step 2: Registering a user

function register_user(string $email, string $password, array &$users):

string { **What this means:**

This function helps you **create a new account**.

It receives:

- $email = the user's email
- $password = the user's password
- &$users = the list where we will save the user

The & means: "Change the original $users list."

Step 2.1: Check if email looks valid

```
if (strpos($email, '@') === false) {
 return "Invalid email format.";
}
```

Simple explanation:

- strpos($email, '@') checks if the email contains @
- If it does not contain @, it says “Invalid email format.”

This is a basic **input check**.

Step 2.2: Check if password is long enough

```
if (strlen($password) < 8) {
 return "Password must be at least 8 characters.";
}
```

Simple explanation:

- strlen() counts the number of characters.
- If the password is shorter than 8, it rejects it.

Longer passwords are harder to guess.

Step 2.3: Save the password securely (MOST IMPORTANT PART) $users[$email] = password_hash($password, PASSWORD_DEFAULT); **What this does in very simple terms:**

Instead of saving the password like:
"mypassword"

It saves a **scrambled version** like:
"$2y$10$W4...random...text..."

Why this is secure:

- If someone steals your data, they won't see real

passwords. • They only see scrambled hashes.

password_hash() also adds a **salt** automatically.
A salt is extra random text that makes hashes harder to crack.

Step 2.4: Show success

return "Registered successfully.";

It just tells you: "Done! User was saved."

Step 3: Logging in a user

function login_user(string $email, string $password, array $users):

string { **What this function does:**

It checks:

- Is this email registered?
- Is the password correct?

Step 3.1: Check if the email exists + password matches

```
if (isset($users[$email]) && password_verify($password,
$users[$email])) {  return "Login successful.";
}
return "Invalid login.";
```

Let's simplify this:

Part A: isset($users[$email])

Means:
"Does this email exist in the list?"

Part B: password_verify(...)

Means:
"Does the typed password match the saved scrambled

password?” Important:

- It does NOT “unscramble” the password.
- It checks by hashing the typed password and comparing safely.

Why it returns “Invalid login.” always (even if email is missing) This is a security trick.

If you say:

- “Email not found” → hacker learns the email is not registered
- “Wrong password” → hacker learns email exists

So we say only:
“Invalid login.”

This prevents **email guessing attacks**.